MPC-PKS – PC02: Komunikace protokolem TCP

Úvod

Protokolová sada TCP/IP obsahuje dva protokoly, které realizují služby pro aplikační vrstvu: TCP a UDP. Protokol TCP zajišťuje spojovanou službu, tj. službu která mezi dvěma aplikacemi naváže spojení. Tento spoj je plně duplexní, data se přenášejí současně a na sobě nezávisle oběma směry. Přenášené segmenty jsou číslovány. Ztracené nebo poškozené segmenty jsou znovu vyžádány.

Protokol TCP vytváří aplikacím iluzi spojitého přenosového proudu, který přenáší jednotlivé bajty. Protokol TCP v zásadě vytváří „přenosovou rouru“ (stream), do které jedna aplikace z jedné strany postupně vkládá jednotlivé bajty, a druhá aplikace je z druhé strany zase bajt po bajtu odebírá. Ve skutečnosti se data samozřejmě nepřenášejí po jednotlivých bajtech, ale po celých blocích, vkládaných do IP datagramů - potřebné členění na bloky však zajišťuje protokol TCP ve své režii a zcela transparentně. Konkrétně to protokol TCP dělá tak, že pracuje s vhodně velkou vyrovnávací pamětí, do které postupně ukládá jednotlivé byty. Jakmile je buffer naplněn, je jeho obsah odeslán jako blok (vložený do IP datagramu), a na druhé straně je vybalen a umístěn do obdobného bufferu, ze kterého si příslušná aplikace pak již může odebírat data. Pokud aplikace komunikují pomocí zasílání „zpráv“, pak si je musí samy extrahovat z přenášeného streamu. Velmi často proto komunikace probíhá v textovém režimu, kde je vhodným oddělovačem zpráv (dvoj)znak konce řádku (\r\n).

Vzhledem k tomu, že si obě komunikující strany potvrzují přijatá data, je spojení TCP možné vždy jen jako bod-bod.

Zadání

Seznamte se s programováním síťové komunikace pomocí protokolu TCP (RFC 793) v prostředí jazyka Python. Vytvořte programy pro vysílání a příjem.

Postupujte v těchto krocích:

1. Vytvořte jednoduchý skript, který otevře spojení TCP na „zrcadlový“ server a sledujte chování komunikace pomocí programu Wireshark pro různé modifikace kódu.

2. Vytvořte skript, který realizuje víceuživatelský TCP terminálový server. Funkčnost ověřte připojením pomocí terminálu PuTTY. Implementujte příkazy **ID**, **PC** a **QUIT**, které vypíší vaše VUT ID, číslo počítače a ukončí spojení.

Cvičení je za 4 body.

Návod ke cvičení MPC-PKS – PC02

ad 1. TCP a časování

a) Zrcadlový server (soubor *01-SERVER.py* v eLearningu, běží na učitelském PC)

„Zrcadlový“ server je spuštěn na učitelském PC. Vyučující sdělí IP adresu. Číslo portu je vždy 50000. Níže je uveden výpis skriptu pro server, který využívá komponentu „socketserver“ pro velmi jednoduchou realizaci vícevláknového TCP serveru. Každé spojení TCP musí na serveru běžet jako paralelní vlákno, viz bod č. 2.

```
import socketserver

# Constants
LISTEN_IP = "0.0.0.0"
LISTEN_PORT = 50000

# Mix TCPServer class with the threading functionality
class ThreadedTCPServer(socketserver.ThreadingMixIn, socketserver.TCPServer):
    pass

class ConnectionHandler(socketserver.StreamRequestHandler):
    # This function is called for every new connection
    def handle(self):
        clientIP, clientPort = self.client_address
        print(f"New client {clientIP}:{clientPort} connected")

        while True:
            try:
                data = self.request.recv(1024)
                if not data:
                    # If the data variable is empty, it means that the client disconnected
                    print(f"Client {clientIP}:{clientPort} disconnected")
                    break # Exit the loop

                # Send back the modified data to the client
                response_data = data.upper()+b"|"
                self.request.sendall(response_data)
                print(f"{clientIP}:{clientPort} - {data}->{response_data}")

            except Exception as e:
                print(f"Client {clientIP}:{clientPort} connection lost")
                break # Exit the loop

# Start of the code
if __name__ == "__main__":
    server = ThreadedTCPServer((LISTEN_IP, LISTEN_PORT), ConnectionHandler)
    print(f"Listening on TCP port {LISTEN_PORT}")
    server.serve_forever()
```

Řádek

```
data = self.request.recv(1024)
```

načte obsah přijímacího bufferu do proměnné `data`. Pokud je buffer prázdný, tak funkce `recv` blokuje provádění vlákna. Načten může být obsah jednoho nebo více paketů podle četnosti volání funkce, maximálně však délka uvedená jako parametr (zde 1024 B). Server převede znaky abecedy na velká písmena, blok doplní znakem „|“ a odešle zpět.

```
response_data = data.upper()+b"|"
self.request.sendall(response_data)
```

Funkce `sendall` zajistí, na rozdíl od funkce `send`, že bude odeslán celý buffer. Např. pro načtený blok ve tvaru `"Ahoj"` bude odesláno zpět `"AHOJ|"`.

b) Skript pro klienta

Základní práce se sokety TCP je obdobná soketům UDP (viz cvičení PC01). Jako základ pro řešení lze použít následující skript.

```
import socket
import time

#adresa a port serveru
host = "127.0.0.1"
port = 50000

#vytvoreni socketu
tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

#otevrit kanal
tcp_socket.connect((host, port))

#zaslani dat (odpoved je ukladana do bufferu)
msg = "data0"
tcp_socket.sendall(msg.encode('ascii'))

#prijem
data = tcp_socket.recv(1024)
print(data)

#uzavreni kanalu
tcp_socket.close()
```

```
tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Konstruktor `socket` vytvoří objekt `tcp_socket`. Prvním parametrem je definice typu adres (AF = Address Family). Druhý parametr definuje typ soketu.

`AF_INET` – adresní prostor IPv4. Protokol IPv6 by byl definován jako `AF_INET6`.
`SOCK_STREAM` – spolehlivá „proudovou“ služba realizovaná protokolem TCP.

```
tcp_socket.connect((host, port))
```

Funkce `connect` naváže spojení TCP s protistranou. Pokud volání proběhne bez chyby je od tohoto místa možné číst a zapisovat do duplexního kanálu. Na rozdíl od UDP není nutné při každém volání funkce pro odesílání vysílání specifikovat adresu příjemce, protože TCP je spojovaná služba a spojení již bylo navázáno.

```
tcp_socket.sendall(msg.encode('ascii'))
data = tcp_socket.recv(1024)
```

Funkce `sendall` vloží celý obsah bufferu (zde textová zpráva `msg` převedená na bajtové pole) do odesílací fronty TCP. K samotnému odeslání paketu může dojít později, např. až se fronta dostatečně zaplní. Funkce `recv` vrací obsah přijímacího bufferu (až do maximální uvedené délky). Pokud je buffer prázdný, tak funkce blokuje provádění programu.

Modifikujte skript tak, aby odeslal např. 5 „zpráv“ typu „data0“, „data1“, atd. s využitím smyčky `for`.

Úkoly:

I. Pozorujte komunikaci, když smyčka `for` obsahuje pouze funkce pro odesílání a příjem.
II. Zjistěte k čemu dojde, pokud smyčka `for` obsahuje také funkce pro otevření a uzavření socketu.

ad 2. TCP server - textová komunikace

Následující výpis ukazuje možnou implementaci TCP serveru pomocí přímé práce se sokety.

Nebude použito!

```
import socket
import time

#prijem na vsech rozhranich a portu 50000
host = ""
port = 50000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind((host, port))

server.listen()

conn, addr = server.accept()
print("Connected by", addr)

while True:
    data = conn.recv(1024)
    if not data: break
    conn.sendall(data)
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Vytvoření objektu typu `socket`, viz bod 1.

```
server.bind((host, port))
```

„Svázání“ soketu s určitou místní adresou a místním portem. Místní počítač může mít několik IP adres, ze kterých vybíráme první parametr. Pokud zadáme prázdný řetězec "" nebo "0.0.0.0", tak bude soket přijímat na všech adresách síťových rozhraní daného počítače. Druhá položka udává místní port. Funkci `bind` je nutné použít, pokud realizujeme službu (server), který přijímá na konkrétním čísle portu.

```
server.listen()
```

Převede soket do režimu, kdy čeká na nové spojení (od klienta).

```
conn, addr = server.accept()
```

Funkce čeká na nové spojení. Vrací objekt `conn` typu `socket`, který se použije pro čtení a zápis do TCP kanálu. Jako druhý parametr vrací adresu klienta ve formě *tuple* (IP, port) pro IPv4.

```
while True:
    data = conn.recv(1024)
    if not data: break
    conn.sendall(data)
```

Ve smyčce se pomocí funkce `recv` čte TCP kanál a v tomto příkladu se funkcí `sendall` „zrcadlí“ data nazpět. Pokud funkce `recv` vrátí prázdné pole, bylo spojení ukončeno klientem.

Základní nevýhodou uvedeného „serveru“ je, že může zpracovávat pouze jedno spojení s klientem. Bylo by nutné opakovaně volat funkci `accept` pro otevírání dalších spojení, což při čekání ve funkci `recv` jednoduše nejde.

Knihovna „socketserver“ obsahuje podporu pro jednoduchou tvorbu serveru s možností připojení více klientů. Uživatel musí pouze definovat funkci, která obsluhuje již otevřené spojení, tedy to, co následuje po funkci `server.accept()` ve výše uvedeném příkladu. Následující výpis ukazuje kostru řešení „serveru“.

```
import socketserver

class MyTCPHandler(socketserver.StreamRequestHandler):

    def handle(self):
        try:
            print("[{}]: open".format(self.client_address[0]))
            self.wfile.write(b"--- PKS server ---\r\n")
            while True:
                line = self.rfile.readline()
                if not line: #klient uzavrel spojeni
                    break
                sline = line.rstrip().upper()
                print("[{}]: RX: {}".format(self.client_address[0], sline))
                #zpracovani prikazu
                match sline:
                    case b'HELP' | b'?':
                        self.wfile.write(b"commands:\r\n")
                        self.wfile.write(b"  help\r\n")
                        self.wfile.write(b"  quit\r\n")
                    case b'......':
                    ...............
        except Exception as e:
            print("[{}]: ER: {}".format(self.client_address[0], e))
        print("[{}]: disconnected".format(self.client_address[0]))

if __name__ == "__main__":
    HOST, PORT = "", 50000
    server = socketserver.TCPServer((HOST, PORT), MyTCPHandler)
    server.serve_forever() #lze prerusit pomoci Ctrl-C
```

Kromě adresy je argumentem konstruktoru serveru třída `MyTCPHandler` (=sada funkcí pro obsluhu spojení). Funkce `serve_forever()` představuje hlavní smyčku serveru a v této jednoduché aplikaci ji lze přerušit pomocí Ctrl-C.

Ve třídě `MyTCPHandler` je v podstatě nutné jen definovat funkci `handle`, která jako parametr `self` dostane soket již otevřeného spojení s klientem. Pokud se k serveru připojuje více klientů, tak je funkce `handle` pokaždé zavolána v novém vláknu. Při opuštění funkce je spojení automaticky uzavřeno. Proto je v těle této funkce obyčejně smyčka, která musí být ukončena při uzavření spojení klientem (přečtou se prázdná data) a dále např. po zadání určitého příkazu.

V těle funkce `handle` je možné využívat pro čtení a zápis standardní funkce soketu

```
data = self.request.recv(1024)
self.request.sendall(.....)
```

Zde je však problematická funkce `recv`, která neposkytuje podporu pro dělení datového proudu na řádky v případě komunikace v textovém formátu.

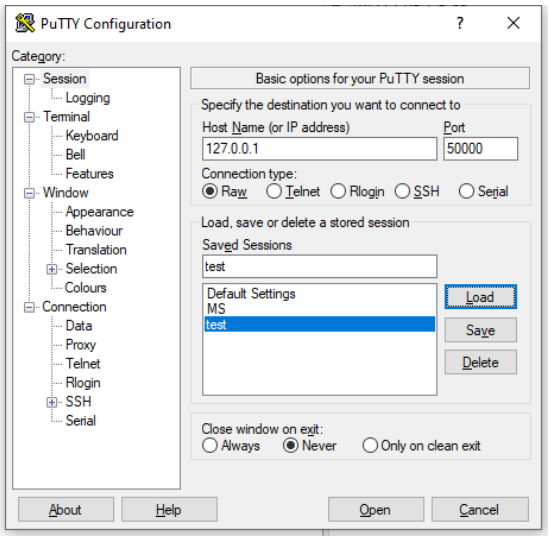
Druhou možností je využívat „souborové“ rozhraní soketu

```
line = self.rfile.readline()
self.wfile.write(......)
```

Funkce `readline()` čte TCP stream až narazí na znak konce řádku, pak vrací přečtený řetězec. Tím se komunikace stává nezávislou na dělení přenášených dat na bloky vkládané do paketů IP.

Umazání případných bílých znaků na začátku a konci řetězce a převod na velká písmena je provedeno pomocí standardních funkcí řetězců

```
sline = line.rstrip().upper()
```

Server může ukončit spojení s klientem pokud dojde k návratu z funkce `handle(self)` použitím příkazu `return`.

Pro ověření funkce serveru je možné použít terminál PuTTY v „raw“ režimu, kdy pracuje jako čistě textový terminál bez řídicích znaků protokolu Telnet. Pro opakované použití terminálu je vhodné si uložit nastavení („Saved Sessions“).